CSR Agents as DAGs

How we build customer-service agents, and how we make them reliable

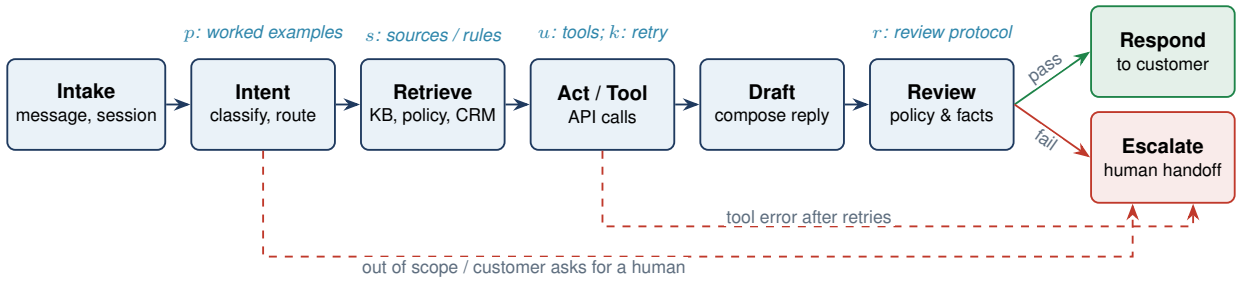
Architecture One-Pager

What we build

Most of our agents are **CSR (customer service representative) bots**: conversational agents that take a customer request from first message to resolution. They understand the request, look up policy and account data, take actions through internal APIs (refunds, order changes, bookings), and either answer the customer or hand off to a human with full context. They talk to real customers, so **predictable and auditable behavior matters more than open-ended autonomy**.

Architecture: a directed acyclic graph (DAG) per conversation turn

Each agent is a **DAG**. **Nodes** are discrete, single-purpose steps; **edges** fix the order of execution and the conditions for branching. The LLM does the work inside a node; the graph decides what happens next.



Typical CSR pipeline. Blue labels show which education component $\theta = (s, p, u, r, k)$ applies at each node. Retries are bounded, so we unroll them into fixed copies of the node and the graph stays acyclic.

Why DAGs instead of free-form autonomous loops

- **Deterministic control flow:** the same inputs follow the same path, with no unbounded "think–act" loops.
- **Testability per node:** each node has its own input/output contract and its own eval set.
- **Easy auditing:** every turn produces a trace of the nodes visited, data retrieved, and tools called.
- **Safe retries:** retries are scoped to one node and capped; side-effecting tools use idempotency keys.
- **Clear escalation paths:** handoff is an explicit edge, not something the model has to decide to do.

Comparison

	DAG agent	Autonomous loop
Control flow	Fixed, reviewable	Model decides each step
Testing	Unit eval per node	End-to-end only
Audit trail	Node-level trace	Long free-form transcript
Failure mode	Local, bounded	Can compound or loop
Cost/latency	Predictable	Varies widely
Escalation	Explicit edge	Depends on the model

Reliability framing: reliability as an education optimization problem

This framing comes from our research repo (`README.md`, `designing_ai_reliability_as_an_education_optimization_problem.md`). We treat the foundation model as a **fixed student**: we do not change its weights. Instead we optimize the **education vector** $\theta = (s, p, u, r, k)$ that we give it:

$$\theta^\star = \arg\max_{\theta} \; \mathrm{LB}(\theta) - \lambda \hat{V}(\theta)$$

subject to a cost budget and **zero policy violations**. Here LB is the Wilson lower bound on the strict-pass rate on unseen tasks, and $\hat{V}$ is the variance across runs. The DAG is what makes this practical: each component of θ maps to a specific node that we can change and measure on its own.

θ	Meaning → CSR equivalent
s **Sources**	Reference rules / handbook → policy KB, refund rules
p **Practice**	Worked examples → few-shot resolved tickets
u **Tools**	Executable checks, retrieval → CRM, order and booking APIs
r **Review**	Self-check before submitting → policy/compliance node
k **Retry**	Retakes on feedback → bounded node retries

What the research shows so far: most failures are *compliance* failures (breaking a rule) rather than *capability* failures. Explicit sources (s) and an automated policy-check retry (k) fix most of them cheaply; in the paper's factorial study, retries rescued 17 of 17 compliance failures. For CSR agents this means investing first in the Retrieve, Review, and retry parts of the graph.

Source: `docs/csr_dag_agent_onepager.tex` in cosmosanalytics/AI-reliability-as-education-optimization. See the full manuscript for the formalism, the experimental design, and its limitations.